AccessMatrix HashiCorp Vault Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AccessMatrix HashiCorp Vault Implementation Guide

HashiCorp Vault is an identity-based secrets and encryption management system. Such secrets may include API encryption keys, passwords, or certificates. Vault provides encryption services that are gated by authentication and authorization methods, manages secrets and protects sensitive data with centralized key management and simple APIs for data encryption. It also performs cryptographic operations by securing secrets such as tokens, passwords, certificates and encryption keys.

Intended Reader

The reader must possess some familiarity with Vault concepts and some knowledge of Vault Command-Line Interface (CLI).

Document Scope

This document covers the integration of AccessMatrix with HashiCorp Vault for the following scenarios:

- Use HashiCorp Vault as key provider for AccessMatrix
- Use HashiCorp Vault to manage AccessMatrix database credentials

While this document provides necessary steps to configure and setup HashiCorp Vault, it will not provide documentation on operating the HashiCorp Vault that already has its own vast and complete documentation. However, when needed this documentation provides:

- Links to the HashiCorp documentation.
- Vault command instructions for resources that are used by AccessMatrix will be provided.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *AccessMatrix Common Deployment Guide*
- *UAS Pseudo E2EEA Implementation Guide*

2. Introduction to AccessMatrix HashiCorp Vault Implementation

This document shows you how to implement and integrate AccessMatrix with HashiCorp Vault.

Terminology

This table lists some common terms used in this documentation. It is not intended as a comprehensive definition for each item.

Term	Description
Vault Seal / Unseal	Vault Server starts in a sealed state. To access full functionality, it has to be unsealed. Unsealing a vault is the process of obtaining the plaintext master key necessary to read the decryption key to decrypt the data.
Shamir Seals	A default configuration of Vault uses Shamir Seals to split keys into shards. A certain threshold of shards is required to reconstruct an unseal key, which is used to decrypt a master key.
Vault Token	Authentication to Vault Server works by verifying the user's identity and generating a token to associate it with the user. There are several types of tokens, mainly root tokens, service token etc.
Vault Secrets Engine	Components that store, generate or encrypt data. e.g. "Transit" or "Database".
Key Provider	It is a key manager in AccessMatrix to define communication interface with HashiCorp Vault server for different implementations of cryptography key provisioning.
Static Role	In a Vault secrets engine, a role describes an identity with a set of permissions, groups, or policies attached to a user. A static role is a role associated with a static username.
Approle	In HashiCorp Vault, an approle represents a set of Vault policies and login constraints that must be met to receive an authentication token with those policies. It is used as a method of authenticating with Vault-defined roles.

AccessMatrix Integration with HashiCorp Vault

You must unseal HashiCorp Vault server before AccessMatrix server can access to Vault for authentication. For more information, refer to the concept of seals in the HashiCorp official documentation, and learn how to configure Vault server.

Unsealing the Vault

Assuming that the above steps have been completed successfully, you should have obtained several unseal keys. A certain threshold of unseal keys is needed to unseal the vault server.

To start the unseal process, enter the following in the command-line:

```
vault operator unseal
```

You should be prompted to enter the unseal key. With each successful key input, the unseal progress will increase, as it shows with the example below:

Key	Value
Seal Type	shamir
Initialized	true
Sealed	true
Total Shares	5
Threshold	3
Unseal Progress	1/3
Unseal Nonce	228e97ec-452e-24fd-6bec-5a938f20a748
Version	1.4.2
HA Enabled	false

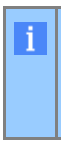
Once the progress is completed, Vault will be unsealed with the following messages:

Key	Value
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	1.4.2
Cluster Name	vault-cluster-e0b76aae
Cluster ID	57651655-4ed3-a5df-8699-dfb8e96096b4
HA Enabled	false

You can now access Vault with the root token initially given during the configuration, with the following command:

```
vault login [root token]
```

After successfully logging into the Vault, all the relevant API will be available to be used for the next few steps.



Note: You may also wish to refer to HashiCorp's documentation on configuring and deploying Vault for detailed information on how to perform the steps above.

3. Using HashiCorp Vault for Key Management

Hashicorp Vault supports both asymmetric and symmetric cryptographic keys and operations. Integration with AccessMatrix only uses symmetric keys. AccessMatrix system uses HashiCorp Vault as a key provider, which means it uses Vault to store symmetric keys and perform cryptographic operations (encryption/decryption) inside it. HashiCorp Vault supports AES-256 symmetric keys with Galois Counter Mode (GCM).

AccessMatrix system does not provide key management to HashiCorp Vault, e.g. creating the keys, destroying the keys, rotating the keys, etc. All of those key management/operations are to be done using either using Vault API or command line.

Out of Integration Scope

While they might be mentioned in other parts of this document, these items are not part of this integration scope, i.e.

- Key management to HashiCorp Vault, e.g. creating the keys, destroying the keys, rotating the keys, etc.
- Asymmetric keys and operations are out of scope in this integration document.

Implementation Checklist

See below a list of items that must be prepared before integrating AccessMatrix with HashiCorp Vault.

Item	Description
Install Vault	You have installed Vault first on your machine, and then verifying the installation. **Note:** AccessMatrix supports vault version v1.3.1 and above.
Unseal Vault	You have unsealed Vault before integrating AccessMarix to it for authentication. Refer to Unsealing the Vault for more details.
Transit Secrets Engine	A secrets engine of the transit type is created. Refer to Create Transit Secrets Engine for more details.
AccessMatrix	You have set up the above items and started Vault, as well as pre-created a secrets engine and a transit key in Vault. Ensure the ports needed for accessing AccessMatrix are opened, for example, by default, TCP/8080 (http) and TCP/8443 (https).

Note: The HashiCorp Vault Transit Key Provider implementation in AccessMatrix does not require external libraries; only AccessMatrix libraries are used.

Implementation Planning

Security Consideration

As described in Performance Consideration section below, AccessMatrix can use HashiCorp Vault Transit key as wrapping key, to unwrap keys that are used by AccessMatrix to secure attributes (AEK) or PIN (PEK). Which means, during operation, the AEK or PEK will be in the clear format in AccessMatrix memory.

To be more secure, you can consider using Vault keys directly, which, on the other hand, will incur some performance penalty.

Integration Consideration

As of version 5.6.7-GA, AccessMatrix supports Vault version v1.3.1 and above.

Performance Consideration

AccessMatrix can use the HashiCorp Vault Transit keys directly, for example as the attribute encryption key (AEK), where AccessMarix calls VaultAPI to encrypt an attribute directly. Thus, every time AccessMatrix needs to have the attribute in clear text, it will call the VaultAPI to decrypt the attribute.

Another option is to use the Vault Keys as wrapping keys, In this case, you can create an AEK in AccessMatrix with the default System Key Provider. This AEK will then be wrapped by the Vault Key. Thus AccessMatrix only needs to call the VaultAPI once to unwrap the AEK. Then this AEK can be used to encrypt/decrypt attributes.

Depending on the use case, for example, if you use it for PseudoE2EEA login module, for faster performance, you might want to consider using the Vault Keys as wrapping keys instead of PIN Encryption Key (PEK) directly.

3.1. AccessMatrix HashiCorp Vault Service

Granting Access to Token with Vault Policies

If you are not using the default **root** token policy with default permissions, you might encounter Permission Denied requests due to lack of privileges.

The following is an example **policy.hcl** configuration file to add the required policy needed to do encryption/decryption in HashiCorp Vault:

KeyOperations.hcl

Code

```
# List, create, update, and delete key/value secrets
path "secret/*"
{
  capabilities =["create","read","update","delete","list"]
}
```

Tokens have to be associated with the relevant policies at creation time as there is no way to edit a token's current policy and the only solution is to revoke a token and create a new token:
`vault token create -policy=KeyOperations`

For more information, refer to Vault documentation on policies.

Create Transit Secrets Engine

For the integration between HashiCorp Vault and AccessMatrix to work, you must create a Transit Secrets Engine in Vault. A secrets engine handles cryptographic functions on data-in-transit. There are many secrets engine that HashiCorp supports and provides, but AccessMatrix uses a particular secrets engine called "Transit". For more details, refer to HashiCorp's documentation on Transits Secret Engine.

Enable Transit Secrets Engine

To create a transits secret engine, run the following in the command line:
`vault secrets enable transit`

This will create a transits secret engine with the default path of transit/. If it is necessary to configure the path, use the `-path` attribute to define your own custom path.

If successful, you will see the following message:

```
Success! Enabled the transit secrets engine at: transit/
```

Create Encryption Key in Vault

To use cryptographic functions, you need to create a key in Vault using the command `vault -write -f transit/keys/my-key` , where:

- `transit/keys` is the default path for transit keys
- `my-key` is the name of the encryption key

This key will also be referenced in the AccessMatrix Console later.

> **Important Note**: Key Derivation in Vault is supported and is false by default. If enabled, all encryption and decryption requests to this key must provide a context to be used. Testing of key info and viewing the KCV currently does not provide context, so it will fail. The use case like PseudoE2EEA Login, the UUID is set as a context in a key wrapper, thus it will function as expected.
> `vault -write -f transit/keys/my-key derived=true`
>
> For more information, refer to Vault API reference.

Verify Key Readiness

Before you start any configuration on the AccessMatrix server, you can verify that the key is ready by running a simple encryption/decryption in the command line:

`vault write transit/encrypt/[key-name]plaintext=aGVsbG8gd29ybGQ=`

To get an output ciphertext like the following:

```
ciphertext vault:v1:oIxxu0SJ0gE6TMqxJ8tekeaD9yC7qjZzWcm6yZu5as/tFyOHHa
```

Decrypt the value with:

```
vault write transit/decrypt/my-key
ciphertext=vault:v1:oIxxu0SJ0gE6TMqxJ8tekeaD9yC7qjZzWcm6yZu5as/tFyOHHa
```

Check the content of the output, and if the output is the same as the original plaintext, then HashiCorp resources configuration is accomplished.

3.2. AccessMatrix HashiCorp Vault Transit Key Provider Deployment

This section assumes that AccessMatrix has been deployed properly on a host machine, and necessary ports have been opened, for example, TCP/8080 (http) and/or TCP/8443 (https). You can confirm it by opening Admin Console using *http://[hostname:port]/am5* (or *https://[hostname:port]/am5*).

Deploy HashiCorp Vault Transit Key Provider Plugin

HashiCorp Vault Transit Key Provider is implemented as a plugin. Thus, you need to deploy the plugin into the AccessMatrix folder by following these steps:

1. Copy **am-hashicorpvtkprov-core.jar** file from the <*ACMATRIX_ROOT*>*sdk**hashicorpvtkprov*\ folder to the <*ACMATRIX_ROOT*>*tomcat**webapps**am5**WEB-INF**lib*\ folder.
2. Copy **am-hashicorpvtkprov-plugin.jar** file from the <*ACMATRIX_ROOT*>*sdk**hashicorpvtkprov*\ folder to the <*ACMATRIX_ROOT*>*tomcat**plugins*\ folder.
3. Copy **field-definitions-hashicorpVaultTransitKeyProvider.xml** from <*ACMATRIX_ROOT*>*sdk**hashicorpvtkprov**res*\ folder to <*ACMATRIX_ROOT*>*tomcat**webapps**am5**WEB-INF**classes*\ folder.
4. Copy **wpecustom-hashicorpVaultTransitKeyProvider.properties** from <*ACMATRIX_ROOT*>*sdk**hashicorpvtkprov**res*\ folder to <*ACMATRIX_ROOT*>*tomcat**webapps**am5**WEB-INF**classes*\ folder.
5. Rename **wpecustom-hashicorpVaultTransitKeyProvider.properties** in the folder <*ACMATRIX_ROOT*>*tomcat**webapps**am5**WEB-INF**classes*\ to **wpecustom.properties**.
6. Add the following line to the file **amsystem.properties** in the folder <ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes\:
   am.field_definition.file.0=field-definitions-hashicorpVaultTransitKeyProvider.xml
7. Start AccessMatrix server, check the **am.log** file (e.g. **<hostname>_am.log**) to see if HashiCorp Vault Transit Key Provider plugin is loaded. Find this keyword "Plugin loaded: hashicorpVaultTransitKeyProvider", e.g.

```
2020-06-03 11:32:52.229 [main] INFO - (HashicorpVaultTransitKeyProvider.java.java:28) Plugin loaded: HashicorpVaultTransitKeyProviderPlugin - 0.1
```

3.3. AccessMatrix HashiCorp Vault Key Provider and Encryption Key Configuration

This part assumes the AccessMatrix HashiCorp Vault Transit Key Provider plugin has been deployed/installed successfully. You can confirm it by navigating to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**, click **Create** button and see if "HashiCorp Vault Transit Key Provider" is listed in the **Implementation** drop-down list.

You should have completed the above, unsealed the vault, and prepared the necessary information to configure the key provider and encryption keys in AccessMatrix to use the HashiCorp Vault Transit Key Provider.

Setup Steps

This section shows you how to configure the AccessMatrix system to integrate with HashiCorp Vault in the Admin Console. Before you start, it is assumed that you have completed all the necessary prerequisites.

- **Configure HashiCorp Vault Transit Key Provider**
  Create a key provider of HashiCorp Vault Transit Key Provider implementation. Refer to *AccessMatrix Common Administration Guide - Configure Key Provider* for details of HashiCorp Vault Transit Key Provider. Use this key provider to integrate with HashiCorp Vault. Refer to Configure HashiCorp Vault Transit Key Provider for more details.
- **Configure Encryption Key**
  AccessMatrix HashiCorp Vault Transit Key Provider supports symmetric keys for the following purposes:
  - Attribute encryption
  - Key derivation
  - Key wrapping
  - Pin encryption
  - Token encryption

Create encryption keys that will be used for various purposes based on the HashiCorp Vault Transit Key Provider created. For example, you create encryption key with "Pin Encryption" purpose to secure the password, or you create encryption key with "Key Wrapping" purpose to encrypt the local encryption

keys (that are defined in Default System Key Provider). Refer to Configure Encryption Key for more details.

> **Notes**:
> - Refer to *AccessMatrix Common Administration Guide - Configure Encryption Key*, for an explanation on each key's purpose.
> - If the use case is for PseudoE2EEA, refer to UAS Pseudo E2EEA Implementation.

Configure HashiCorp Vault Transit Key Provider

After successfully deploying the AccessMatrix HashiCorp Vault Transit Key Provider plugin, the key provider implementation will be available in the AccessMatrix Admin Console.

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider** on the left pane.
2. Click **Create** and select "HashiCorp Vault Transit Key Provider" in the **Implementation** drop-down list.
3. Click **Next**.
4. Enter a **Key Provider Id** (e.g. "hashiCorpVKey") and **Key Provider Name** (e.g. "hashiCorpVKey") and enter an optional description.
5. Specify the **API Url** (e.g "https://10.1.42.201/8201/v1")
6. Optionally, specify the **Secret Engine Path** (e.g "/transit")
7. Specify the **Vault Token** (e.g "s.iyNUhq8Ov4hIAx6snw5mB2nL")
8. Test the connection by clicking the **Test Connection** button below the implementation name.
9. Click **Save**. A new key provider is created.

Configure Encryption Key

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key** on the left pane.
2. Click **Create** and select the newly created **Key Provider** (e.g. "hashiCorpVKey") from the **Key Provider** drop-down list. Notice that there is only one available **Encryption Key Template**, "Symmetric Key (secret key)", as AccessMatrix System integrates with HashiCorp Vault to perform symmetric operations only.

3. Click **Next**.
4. Enter an **Encryption Key Id** (e.g "HCEncKey") and an **Encryption Key Name** (e.g. "Vault Encryption Key") and enter its description.
5. If the connection to HashiCorp Vault is established, AccessMatrix will automatically fetch the transit key name(s) stored in Vault, and then display the list of transit key names in the drop-down list for selection.
6. Next, select a key purpose from **Purpose** drop-down list on the **Attributes** tab.
7. Click **Save**. An encryption key is created.

Notes:

- You can click the **Test KeyInfo** and **View Public Key** buttons to check if the key was successfully created.
- If the Key Derivation is set in Hashicorp Vault, the **Test KeyInfo** and **View Public Key** buttons will not function. Refer to Create Encryption Key in Vault for more details.

4. Using HashiCorp Vault for Database Credential Management

HashiCorp Vault supports the management of database credentials and can be integrated with the AccessMatrix databse to programmatically retrieve and rotate the AccessMatrix database password.

In this implementation, a static role associated with an AccessMatrix database user's credentials is defined within a database secrets engine. The approle method is used to authenticate with HashiCorp Vault to retrieve and rotate the database credentials.

This guide describes the integration of a MySQL database with HashiCorp Vault.

> **Note**: It is assumed that the database has already been created and configured to connect with AccessMatrix. Refer to *AccessMatrix Common Deployment Guide - AccessMatrix Installation* for more details on setting up the AccessMatrix database.

Implementation Checklist

The following is a checklist of the steps needed to integrate HashiCorp Vault with AccessMatrix:

Task	Reference
Install Vault on your machine and verify the installation	Install Vault
Initialize and unseal the Vault	Unseal the Vault
Create Vault policies to grant access to AccessMatrix database	Create Vault Policies
Define Vault environmental variables	Define Environmental Variables
Enable and configure database secrets engine	Create and Set Up Database Secrets Engine
Create static role	Create Static Role
Create and enable approle	Create and Enable AppRole
Authenticate to Vault using approle method	Authenticate to Vault using AppRole
Encrypt and update database password in am5.xml	Update Database Password in am5.xml

Integration Consideration

As of version 5.7.4-GA, AccessMatrix supports Vault version v1.12.2 and above.

4.1. Configure and Deploy HashiCorp Vault

Create Vault Policies

If you are not using the default **root** token policy with default permissions, you might encounter Permission Denied messages due to a lack of privileges. HashiCorp Vault policies allow you to define the access permissions to specific paths and operations within Vault.

The following steps describe how to create and upload the Vault policy files required to perform the integration between AccessMatrix and HashiCorp Vault for database credential management.

1. Create and save a Vault policy file named **readonly.hcl** containing the read permission for reading the AccessMatrix database user's static credentials in Vault (which will be configured in a later section):

readonly.hcl

Code

```
path "database/static-creds/amdb-reader"{
  capabilities =["read"]
}
```

2. Write the "amdb-readonly" policy for HashiCorp Vault by uploading the previously created hcl file. To do so, enter the following in the command-line:

   `vault policy write amdb-readonly readonly.hcl`

3. Create and save a Vault policy file named **amdb-tables.hcl** containing the relevant permissions to access the AccessMatrix database path in Vault (which will be configured in a later section):

amdb-tables.hcl

Code

```
path "database/data/amdb/*"{
  capabilities =["create","read","update","delete"]
}

path "database/*"{
  capabilities =["deny"]
}
```

4. Write the "amdb-tables" policy for HashiCorp Vault by uploading the previously created hcl file. To do so, enter the following in the command-line:

```
vault policy write amdb-tables amdb-tables.hcl
```

Define Environmental Variables

> **Note**: Throughout this guide, it is assumed that the IP addresses and ports below are used. Replace these values while following the steps as needed.
>
> - Vault server - 127.0.0.1:8200
> - AccessMatrix Database - 192.168.50.103:3306

The following steps describe how define environmental variables for Vault using the Vault Command-Line Interface (CLI).

> **Note**: Replace `export` with `set` if you are using Windows.

1. Define an environment variable for the vault CLI to address the server:

   ```
   export VAULT_ADDR=http://127.0.0.1:8200
   ```

2. Define an environment variable for the vault CLI to authenticate with the Vault server:

   ```
   export VAULT_TOKEN=root
   ```

3. Define an environmental variable for the AccessMatrix Database address:

   ```
   export DB_URL=192.168.50.103:3306
   ```

Create and Set Up Database Secrets Engine

For the integration between HashiCorp Vault and AccessMatrix to work, you must create a database secrets engine in Vault. The following steps describe how to create and configure the database secrets engine using the Vault CLI.

> **Note**: For more details, refer to HashiCorp's documentation on the database secret engine.

1. In your AccessMatrix database, create a root user specifically for Vault operations. For example:

   MySQL

   ```
   CREATE USER 'vaultuser'@'%' IDENTIFIED BY 'vaultpass';
   GRANT ALL PRIVILEGES ON amdb.* TO 'vaultuser'@'%' WITH
   GRANT OPTION;
   GRANT CREATE USER ON *.* to 'vaultuser'@'%';
   FLUSH PRIVILEGES;
   ```

2. Next, create a database secret engine for Vault in the CLI:

   ```
   vault secrets enable database
   ```

If successful, you will see the following message:

```
Success! Enabled the database secrets engine at: database/
```

> **Note**: This creates a database secret engine with the default path of *database/*.

3. Configure the database secrets engine to use the AccessMatrix database:

   ```
   vault write database/config/amdb plugin_name=mysql-database-plugin
   connection_url="{{username}}:{{password}}@tcp($DB_URL)/amdb"allowed_roles="*"usern
   ame="vaultuser"password="vaultpass"
   ```

> **Note**: The values for `username` and `password` at the end of the command should match those of the previously created root user, (e.g. "vaultuser" and "vaultpass" respectively).

Create Static Role

In a secrets engine, a role describes an identity with a set of permissions, groups, or policies attached to a user. In this implementation, the database secrets engine uses a static role (i.e. a role that is associated with a static username) and manages its password for database access, password rotation, etc.

The following steps describe how to create and verify a static role using the Vault CLI.

1. Create and save an SQL file named **rotation.sql** with following contents:

   MySQL

   ```
   ALTER USER "{{name}}" IDENTIFIED BY '{{password}}';
   ```

2. Create a static role for accessing the database with the following command:

   ```
   vault write database/static-roles/amdb-reader db_name=amdb
   rotation_statements=@rotation.sql username="vaultuser"rotation_period=86400
   policies=amdb-tables
   ```

   Replace the values for the following fields as necessary:

   a. `rotation_statements` - This should be the filename of the rotation statement file created in the previous step with the "@" prefix (e.g. "@rotation.sql").

   b. `username` - This should be the username of the root user created in the Create and Enable AppRole section (e.g. "vaultuser").

c. `rotation_period` - This determines how often the password is rotated in seconds (e.g. "86400", which means every 86400 seconds).

d. `policies` - This should be the name of the policy created in the Create Vault Policies section that grants access permissions to the AccessMatrix database (e.g. "amdb-tables").

Create and Enable AppRole

In HashiCorp Vault, an approle represents a set of Vault policies and login constraints that must be met to receive a token with those policies. An approle is used as a method allowing machines or apps to authenticate with Vault-defined roles. Approles must have an least one constraint to be used.

The following steps describe how to create an approle that will be used to authenticate to Vault.

1. Enable authentication via approle in Vault:
   `vault auth enable approle`
2. Create an approle for authentication:
   `vault write auth/approle/role/amdb-approle policies=default,amdb-readonly bind_secret_id=false secret_id_bound_cidrs=192.168.0.0/16`

   Notes:
   - At least one constraint must be imposed for approles. The above command imposes the constraint that only a machine/app from the address range 192.168.0.0/16 is allowed to authenticate to Vault using approle. Replace the value for `secret_id_bound_cidrs` with the appropriate address range.
   - Alternatively, you may choose to enable `bind_secret_id` instead, or to enable both constraints for added security. Setting `bind_secret_id` to "true" imposes the constraint that a SecretID for the approle must be issued and used for successful authentication.
   - Refer to HashiCorp's documentation on approles for more information.

4.2. Retrieve Database Credentials Using HashiCorp Vault

Authenticate to Vault using AppRole

The following steps describe how to authenticate to Vault to retrieve the database password using the approle method.

1. Fetch the RoleID of the previously created approle with the following command in the CLI:
   `vault read auth/approle/role/amdb-approle/role-id`

   > **Note**: If you set bind_secret_id to "true" in the previous section, you should also get the SecretID for your approle using the command:
   > `vault write -f auth/approle/role/amdb-approle/secret-id`

2. Log in to Vault by making an **HTTP POST** request to *http://127.0.0.1:8200/v1/auth/approle/login* as follows, replacing the value of role-id with your approle's RoleID:

Sample Request

JSON

```
{
"role_id" : "fefb85d1-d835-93cb-bbd3-687fca5a12ef"
}
```

Sample Response (Successful)

JSON

```
{
    "request_id": "12d9701d-1a31-5e12-bde4-50393e30b612",
    "lease_id": "",
    "renewable": false,
    "lease_duration": 0,
    "data": null,
    "wrap_info": null,
    "warnings": null,
    "auth": {
        "client_token":
"hvs.CAESIBHWkdYPBIoj80xOiFIie8mauQOToz3kCej_fgREfXKoGh4KHGh2cy5UY0JoZTBpVXp
mcGtDbU5ud3ljTkVkMWY",
        "accessor": "7AHuXtCfmH5haqhorqLSjU0A",
        "policies": [
            "amdb-readonly",
            "default"
        ],
        "token_policies": [
```

Sample Request


```
            "amdb-readonly",
            "default"
        ],
        "metadata": {
            "role_name": "amdb-approle"
        },
        "lease_duration": 2764800,
        "renewable": true,
        "entity_id": "b1fe7968-6300-28d4-39bc-9f4974b9c1d7",
        "token_type": "service",
        "orphan": true,
        "mfa_requirement": null,
        "num_uses": 0
    }
}
```


Note: If you set `bind_secret_id` to "true" in the previous section, you must also provide the SecretID of the approle with the parameter secret_id.

3. Note the value of client_token in the response.
4. Retrieve the AccessMatrix database password by making an **HTTP GET** request to *http://127.0.0.1:8200/v1/database/static-creds/amdb-reader*, including the client token value from the previous step in an X-Vault-Token header. If successful, you should receive a response with the database password:

Sample Response (Successful)

JSON


```
{
    "data": {
        "password" : "A1a-jxH944nG0qHRpMNR",
        "last_vault_rotation" : "2023-01-
17T06:53:03.527967035Z",
        "rotation_period" : "86400",
        "ttl" : "12117",
        "username" : "vaultuser"
    }
    "auth": null,
    "renewable" : false,
    "warnings" : null,
    "lease_duration": 0,
    "request_id" "0d6976ec-3074-2b1c-0265-1e749666e65e",
    "lease_id" : "",
    "wrap_info" : null
            }
}
```

Note that the password has been automatically rotated by Vault and now reflects a different value (i.e. "A1a-jxH944nG0qHRpMNR").

Update Database Password in am5.xml

To avoid leaving the database password in plaintext, the password should first be encrypted. AccessMatrix comes bundled with the Encryptor Utility tool, which provides various methods of encrypting truststore, keystore, and database passwords. To encrypt the database password:

1. Open the command prompt and run the **encryptor.bat** file (or the **encryptor.sh** file for Linux) from the *<ACMATRIX_ROOT>\bin* folder. A menu will be displayed.
2. Select `HashicorpVaultEncryptor` from the menu by typing "3" and pressing Enter.
3. Provide a salt for the encryptor by selecting any of the options from 1-5 and following the instructions.
4. Select `Encrypt a passphrase` from the next menu by typing "1" and pressing Enter. Follow the instructions, providing the appropriate values as prompted. The encrypted passphrase is generated under `SingleFieldEncryptedValue`.
5. Copy the full encrypted passphrase under `SingleFieldEncryptedValue`.
6. Navigate to *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost* and open **am5.xml**.
7. Under the password field, replace the password with the encrypted passphrase. For example:

XML

```
<Resource name="jdbc/DSamdb" auth="Container" type="javax.sql.DataSource"
description="AM5 DB"
factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10" maxWaitMillis="10000"
maxTotal="80" maxIdle="20" minIdle="10" validationQuery="select 1"
username="vaultuser"
password="HashicorpVaultEncryptor:Tzm4MHyYJnLjUwYMB9l3lP/q2a6Y7qHgvZHhHwR
HJGJEYIjGjmLyxDrz2ol7jrneOIGDbLAL8NUjUWa2ZkP4wCuMBGi3F/wB7Y5vEnDhG4SN/jxJ
rfNNdEZWq2HZwUQEQ2987fL7tk1W+cJYHLmIKGaLl7SslK7tAhHHWMU9hDEOoluRsCVrWeb6x
GyNIkYCl934JrjNUouOopfcnZrluJSm3dEFFFelakXK4oTd7Ro="
driverClassName="com.mysql.jdbc.Driver"
url="jdbc:mysql://10.1.11.85:3306/amdb?useUnicode=true&amp;characterEncod
ing=utf8"
 />
```

The integration between HashiCorp Vault and the AccessMatrix database is complete.

Note: For the detailed steps on using the HashicorpVaultEncryptor in the Encryptor Utility Tool, refer to *AccessMatrix Common Deployment Guide - Using HashicorpVaultEncryptor to Encrypt Database Password*.